Appendix C MATLAB® Examples

These computer examples are written for MATLAB. If you have MATLAB available, you should try some of them on your computer. For information on getting started with MATLAB, see Section C.1. Several functions have been written to allow for experimentation with MATLAB. The MATLAB functions associated with this book are available at

```
bit.ly/2HyvR8n
```

We recommend that you create a directory or folder to store these files and download them to that directory or folder. One method for using these functions is to launch MATLAB from the directory where the files are stored, or launch MATLAB and change the current directory to where the files are stored. In some versions of MATLAB the working directory can be changed by changing the current directory on the command bar. Alternatively, one can add the path to that directory in the MATLAB path by using the *path* function or the Set Path option from the File menu on the command bar.

If MATLAB is not available, it is still possible to read the examples. They provide examples for several of the concepts presented in the book. Most of the examples used in the MATLAB appendix are similar to the examples in the Mathematica and Maple appendices. MATLAB,

however, is limited in the size of the numbers it can handle. The maximum number that MATLAB can represent in its default mode is roughly 16 digits and larger numbers are approximated. Therefore, it is necessary to use the symbolic mode in MATLAB for some of the examples used in this book.

A final note before we begin. It may be useful when doing the MATLAB exercises to change the formatting of your display. The command

```
>> format rat
```

sets the formatting to represent numbers using a fractional representation. The conventional *short* format represents large numbers in scientific notation, which often doesn't display some of the least significant digits. However, in both formats, the calculations, when not in symbolic mode, are done in floating point decimals, and then the rational format changes the answers to rational numbers approximating these decimals.

C.1 Getting Started with MATLAB

MATLAB is a programming language for performing technical computations. It is a powerful language that has become very popular and is rapidly becoming a standard instructional language for courses in mathematics, science, and engineering. MATLAB is available on most campuses, and many universities have site licenses allowing MATLAB to be installed on any machine on campus.

In order to launch MATLAB on a PC, double click on the MATLAB icon. If you want to run MATLAB on a Unix system, type *matlab* at the prompt. Upon launching MATLAB, you will see the MATLAB prompt:

```
>>
```

which indicates that MATLAB is waiting for a command for you to type in. When you wish to quit MATLAB, type *quit* at the command prompt.

MATLAB is able to do the basic arithmetic operations such as addition, subtraction, multiplication, and division. These can be accomplished by the operators +, -, *, and /, respectively. In order to raise a number to a power, we use the operator ^ . Let us look at an example:

If we type $2 \hat{} 7 + 125/5$ at the prompt and press the *Enter* key

```
>> 2^7 + 125/5
```

then MATLAB will return the answer:

```
ans =
   153
```

Notice that in this example, MATLAB performed the exponentiation first, the division next, and then added the two results. The order of operations used in MATLAB is the one that we have grown up using. We can also use parentheses to change the order in which MATLAB calculates its quantities. The following example exhibits this:

```
>> 11*( (128/(9+7) - 2^(72/12)))

ans =
   -616
```

In these examples, MATLAB has called the result of the calculations *ans*, which is a variable that is used by MATLAB to store the output of a computation. It is possible to assign the result of a computation to a specific variable. For example,

```
>> spot=17

spot =
   17
```

assigns the value of 17 to the variable *spot*. It is possible to use variables in computations:

```
>> dog=11

dog =
   11

>> cat=7

cat =
   7

>> animals=dog+cat

animals =
   18
```

MATLAB also operates like an advanced scientific calculator since it has many functions available to it. For example, we can do the standard operation of taking a square root by using the *sqrt* function, as in the following example:

```
>> sqrt(1024)

ans =
   32
```

There are many other functions available. Some functions that will be useful for this book are *mod, factorial, factor, prod*, and *size*.

Help is available in MATLAB. You may either type *help* at the prompt, or pull down the Help menu. MATLAB also provides help from the command line by typing *help commandname*. For example, to get help on the function *mod*, which we shall be using a lot, type the following:

```
>> help mod
```

MATLAB has a collection of toolboxes available. The toolboxes consist of collections of functions that implement many application-specific tasks. For example, the Optimization toolbox provides a collection of functions that do linear and nonlinear optimization. Generally, not all toolboxes are available. However, for our purposes, this is not a problem since we will only need general MATLAB functions and have built our own functions to explore the number theory behind cryptography.

The basic data type used in MATLAB is the matrix. The MATLAB programming language has been written to use matrices and vectors as the most fundamental data type. This is natural since many mathematical and scientific problems lend themselves to using matrices and vectors.

Let us start by giving an example of how one enters a matrix in MATLAB. Suppose we wish to enter the matrix

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 27 \\ 1 & 4 & 16 & 64 \end{bmatrix}$$

into MATLAB. To do this we type:

```
>> A = [1 1 1 1; 1 2 4 8; 1 3 9 27; 1 4 16 64]
```

at the prompt. MATLAB returns

```
A =
    1 1   1    1
    1 2   4    8
    1 3   9    27
    1 4 16   64
```

There are a few basic rules that are used when entering matrices or vectors. First, a vector or matrix is started by using a square bracket [and ended using a square bracket]. Next, blanks or commas separate the elements of a row. A semicolon is used to end each row. Finally, we may place a semicolon at the very end to prevent MATLAB from displaying the output of the command.

To define a row vector, use blanks or commas. For example,

```
>> x = [2, 4, 6, 8, 10, 12]

x =
     2   4   6   8   10   12
```

To define a column vector, use semicolons. For example,

```
>> y=[1;3;5;7]

y =
   1
   3
   5
   7
```

In order to access a particular element of y, put the desired index in parentheses. For example, y(1) = 1, y(2) = 3, and so on.

MATLAB provides a useful notation for addressing multiple elements at the same time. For example, to access the third, fourth, and fifth elements of x, we would type

```
>> x(3:5)

ans =
    6   8   10
```

The 3:5 tells MATLAB to start at 3 and count up to 5. To access every second element of x, you can do this by

```
>> x(1:2:6)

ans =
    2   6   10
```

We may do this for the array also. For example,

```
>> A(1:2:4,2:2:4)

ans =
    1   1
    3  27
```

The notation 1:n may also be used to assign to a variable. For example,

```
>> x=1:7
```

returns

```
x =
    1   2   3   4   5   6   7
```

MATLAB provides the *size* function to determine the dimensions of a vector or matrix variable. For example, if we want the dimensions of the matrix A that we entered earlier, then we would do

```
>> size(A)

ans =
    4   4
```

It is often necessary to display numbers in different formats. MATLAB provides several output formats for displaying the result of a computation. To find a list of formats available, type

```
>> help format
```

The *short* format is the default format and is very convenient for doing many computations. However, in this book, we will be representing long whole numbers, and the *short* format will cut off some of the trailing digits in a number. For example,

```
>> a=1234567899

a =
   1.2346e+009
```

Instead of using the *short* format, we shall use the *rational* format. To switch MATLAB to using the rational format, type

```
>> format rat
```

As an example, if we do the same example as before, we now get different results:

```
>> a=1234567899

a =
   1234567899
```

This format is also useful because it allows us to represent fractions in their fractional form, for example,

```
>> 111/323

ans =
   111/323
```

In many situations, it will be convenient to suppress the results of a computation. In order to have MATLAB suppress printing out the results of a command, a semicolon must follow the command. Also, multiple commands may be entered on the same line by separating them with commas. For example,

```
>> dogs=11, cats=7; elephants=3, zebras=19;

dogs =
   11

elephants =
   3
```

returns the values for the variables *dogs* and *elephants* but does not display the values for *cats* and *zebras*.

MATLAB can also handle variables that are made of text. A string is treated as an array of characters. To assign a string to a variable, enclose the text with single quotes. For example,

```
>> txt='How are you today?'
```

returns

```
txt =
   How are you today?
```

A string has size much like a vector does. For example, the size of the variable txt is given by

```
>> size(txt)
ans =
    1   18
```

It is possible to edit the characters one by one. For example, the following command changes the first word of txt:

```
>> txt(1)='W'; txt(2)='h';txt(3)='o'

txt =
   Who are you today?
```

As you work in MATLAB, it will remember the commands you have entered as well as the values of the variables you have created. To scroll through your previous commands, press the up-arrow and down-arrow. In order to see the variables you have created, type *who* at the prompt. A similar command *whos* gives the variables, their size, and their type information.

Notes. 1. To use the commands that have been written for the examples, you should run MATLAB in the directory into which you have downloaded the file from the Web site `bit.ly/2HyvR8n`

2. Some of the examples and computer problems use long ciphertexts, etc. For convenience, these have been stored in the file `ciphertexts.m`,

which can be loaded by typing *ciphertexts* at the prompt. The ciphertexts can then be referred to by their names. For example, see Computer Example 4 for Chapter 2.

C.2 Examples for Chapter 2

Example 1

A shift cipher was used to obtain the ciphertext `kddkmu`.

Decrypt it by trying all possibilities.

```
>> allshift('kddkmu')


kddkmu
leelnv
mffmow
nggnpx
ohhoqy
piiprz
qjjqsa
rkkrtb
sllsuc
tmmtvd
unnuwe
voovxf
wppwyg
xqqxzh
yrryai
zsszbj
attack
buubdl
cvvcem
dwwdfn
exxego
fyyfhp
gzzgiq
haahjr
ibbiks
jccjlt
```

As you can see, `attack` is the only word that occurs on this list, so that was the plaintext.

Example 2

Encrypt the plaintext message `cleopatra` using the affine function $7x + 8$:

```
>> affinecrypt('cleopatra',7,8)

ans =


'whkcjilxi'
```

Example 3

The ciphertext `mzdvezc` was encrypted using the affine function $5x + 12$. Decrypt it.

SOLUTION

First, solve $y \equiv 5x + 12 \pmod{26}$ for x to obtain $x \equiv 5^{-1}(y - 12)$. We need to find the inverse of $5 \pmod{26}$:

```
>> powermod(5,-1,26)

ans =
   21
```

Therefore, $x \equiv 21(y - 12) \equiv 21y - 12 \cdot 21$. To change $-12 \cdot 21$ to standard form:

```
>> mod(-12*21,26)
```

```
ans =
   8
```

Therefore, the decryption function is $x \equiv 21y + 8$. To decrypt the message:

```
>> affinecrypt('mzdvezc',21,8)

ans =


'anthony'
```

In case you were wondering, the plaintext was encrypted as follows:

```
>> affinecrypt('anthony',5,12)

ans =


'mzdvezc'
```

Example 4

Here is the example of a Vigenère cipher from the text. Let's see how to produce the data that was used in Section 2.3 to decrypt the ciphertext. In the file ciphertexts.m, the ciphertext is stored under the name *vvhq*. If you haven't already done so, load the file ciphertexts.m:

```
>> ciphertexts
```

Now we can use the variable vvhq to obtain the ciphertext:

```
>> vvhq

vvhqwvvrhmusgjgthkihtssejchlsfcbgvwcrlryqtfsvgahw
kcuhwauglqhnslrljs
hbltspisprdxljsveeghlqwkasskuwepwqtwvspgoelkcqyfn
svwljsniqkgnrgybwl
wgoviokhkazkqkxzgyhcecmeiujoqkwfwvefqhkijrclrlkbi
enqfrjljsdhgrhlsfq
twlauqrhwdmwlgusgikkflryvcwvspgpmlkassjvoqxeggvey
ggzmljcxxljsvpaivw
ikvrdrygfrjljslveggveyggeiapuuisfpbtgnwwmuczrvtwg
lrwugumnczvile
```

We now find the frequencies of the letters in the ciphertext. We use the function *frequency*. The *frequency* command was written to display automatically the letter and the count next to it. We therefore have put a semicolon at the end of the command to prevent MATLAB from displaying the count twice.

```
>> fr=frequency(vvhq);
 a   8
 b   5
 c  12
 d   4
 e  15
 f  10
 g  27
 h  16
 i  13
 j  14
 k  17
 l  25
 m   7
 n   7
 o   5
 p   9
 q  14
 r  17
 s  24
 t   8
 u  12
 v  22
 w  22
```

```
x   5
y   8
z   5
```

Let’s compute the coincidences for displacements of 1, 2, 3, 4, 5, 6:

```
>> coinc(vvhq,1)

ans =
   14

>> coinc(vvhq,2)

ans =
   14

>> coinc(vvhq,3)

ans =
   16

>> coinc(vvhq,4)

ans =
   14

>> coinc(vvhq,5)

ans =
   24

>> coinc(vvhq,6)

ans =
   12
```

We conclude that the key length is probably 5. Let’s look at the 1st, 6th, 11th, ... letters (namely, the letters in positions congruent to 1 mod 5). The function *choose* will do this for us. The function *choose(txt,m,n)* extracts every letter from the string txt that has positions congruent to n mod m.

```
>> choose(vvhq,5,1)

ans =

vvuttcccqgcunjtpjgkuqpknjkygkkgcjfqrkqjrqudukvpkv
ggjjivgjgg pfncwuce
```

We now do a frequency count of the preceding substring. To do this, we use the *frequency* function and use ans as input. In MATLAB, if a command is issued without declaring a variable for the result, MATLAB will put the output in the variable ans.

```
>> frequency(ans);


 a   0
 b   0
 c   7
 d   1
 e   1
 f   2
 g   9
 h   0
 i   1
 j   8
 k   8
 l   0
 m   0
 n   3
 o   0
 p   4
 q   5
 r   2
 s   0
 t   3
 u   6
 v   5
 w   1
 x   0
 y   1
 z   0
```

To express this as a vector of frequencies, we use the *vigvec* function. The *vigvec* function will not only display the frequency counts just shown, but will return a vector that contains the frequencies. In the following output, we have suppressed the table of frequency counts since they appear above and have reported the results in the *short* format.

```
>> vigvec(vvhq,5,1)

ans =
   0
   0
   0.1045
   0.0149
   0.0149
   0.0299
   0.1343
   0
   0.0149
   0.1194
   0.1194
   0
   0
   0.0448
   0
   0.0597
   0.0746
   0.0299
   0
   0.0448
   0.0896
   0.0746
   0.0149
   0
   0.0149
   0
```

(If we are working in rational format, these numbers are displayed as rationals.) The dot products of this vector with the displacements of the alphabet frequency vector are computed as follows:

```
>> corr(ans)

ans =
   0.0250
   0.0391
   0.0713
   0.0388
   0.0275
   0.0380
   0.0512
   0.0301
   0.0325
   0.0430
   0.0338
   0.0299
   0.0343
   0.0446
   0.0356
   0.0402
   0.0434
   0.0502
   0.0392
   0.0296
   0.0326
   0.0392
   0.0366
   0.0316
   0.0488
   0.0349
```

The third entry is the maximum, but sometimes the largest entry is hard to locate. One way to find it is

```
>> max(ans)

ans =
   0.0713
```

Now it is easy to look through the list and find this number (it usually occurs only once). Since it occurs in the third position, the first shift for this Vigenère cipher is by 2, corresponding to the letter *c*. A procedure similar to the one just used (using *vigvec(vvhq, 5,2), . . . , vigvec(vvhq,5,5)*)

shows that the other shifts are probably 14, 3, 4, 18. Let’s check that we have the correct key by decrypting.

```
>> vigenere(vvhq,-[2,14,3,4,18])

ans =

themethodusedforthepreparationandreadingofcodemes
sagesissimpleinthe
extremeandatthesametimeimpossibleoftranslationunl
essthekeyisknownth
eeasewithwhichthekeymaybechangedisanotherpointinf
avoroftheadoptiono
fthiscodebythosedesiringtotransmitimportantmessag
eswithouttheslight
estdangeroftheirmessagesbeingreadbypoliticalorbus
inessrivalsetc
```

For the record, the plaintext was originally encrypted by the command

```
>> vigenere(ans,[2,14,3,4,18])

ans =

vvhqwvvrhmusgjgthkihtssejchlsfcbgvwcrlryqtfsvgahw
kcuhwauglqhnslrljs
hbltspisprdxljsveeghlqwkasskuwepwqtwvspgoelkcqyfn
svwljsniqkgnrgybwl
wgoviokhkazkqkxzgyhcecmeiujoqkwfwvefqhkijrclrlkbi
enqfrjljsdhgrhlsfq
twlauqrhwdmwlgusgikkflryvcwvspgpmlkassjvoqxeggvey
ggzmljcxxljsvpaivw
ikvrdrygfrjljslveggveyggeiapuuisfpbtgnwwmuczrvtwg
lrwugumnczvile
```

C.3 Examples for Chapter 3

Example 5

Find $\gcd(23456; 987654)$.

```
>> gcd(23456,987654)

ans =
   2
```

If larger integers are used, they should be expressed in symbolic mode; otherwise, only the first 16 digits of the entries are used accurately. The present calculation could have been done as

```
>> gcd(sym('23456'),sym('987654'))

ans =
   2
```

Example 6

Solve $23456x + 987654y = 2$ in integers x, y.

```
>> [a,b,c]=gcd(23456,987654)

a =
   2

b =
   -3158
```

```
c =
   75
```

This means that 2 is the gcd and $23456 \cdot (-3158) + 987654 \cdot 75 = 2$.

Example 7

Compute $234 \cdot 456 \pmod{789}$.

```
>> mod(234*456,789)

ans =
   189
```

Example 8

Compute $234567^{876543} \pmod{565656565}$.

```
>>
powermod(sym('234567'),sym('876543'),sym('5656565
65'))

ans =
   5334
```

Example 9

Find the multiplicative inverse of $87878787 \pmod{9191919191}$.

```
>> invmodn(sym('87878787'),sym('9191919191'))

ans =
   7079995354
```

Example 10

Solve $7654x \equiv 2389 \pmod{65537}$.

SOLUTION

To solve this problem, we follow the method described in Section 3.3. We calculate 7654^{-1} and then multiply it by 2389:

```
>> invmodn(7654,65537)

ans =
   54637

>> mod(ans*2389,65537)

ans =
   43626
```

Example 11

Find x with

$$x \equiv 2 \pmod{78}, \quad x \equiv 5 \pmod{97}, \quad x \equiv 1 \pmod{119}.$$

SOLUTION

To solve the problem we use the function *crt*.

```
>> crt([2 5 1],[78 97 119])

ans =
   647480
```

We can check the answer:

```
>> mod(647480,[78 97 119])
```

```
ans =
    2   5   1
```

Example 12

Factor 123450 into primes.

```
>> factor(123450)

ans =
    2    3   5   5   823
```

This means that $123450 = 2^1 3^1 5^2 823^1$.

Example 13

Evaluate $\phi(12345)$.

```
>> eulerphi(12345)

ans =
   6576
```

Example 14

Find a primitive root for the prime 65537.

```
>> primitiveroot(65537)

ans =
   3
```

Therefore, 3 is a primitive root for 65537.

Example 15

Find the inverse of the matrix

$$\begin{matrix} 13 & 12 & 35 \\ 41 & 53 & 62 \\ 71 & 68 & 10 \end{matrix} \pmod{999}.$$

SOLUTION

First, we enter the matrix as M.

```
>> M=[13 12 35; 41 53 62; 71 68 10];
```

Next, invert the matrix without the mod:

```
>> Minv=inv(M)

Minv =
     233/2158     -539/8142    103/3165
     -270/2309    139/2015     -40/2171
     209/7318     32/34139    -197/34139
```

We need to multiply by the determinant of M in order to clear the fractions out of the numbers in $Minv$. Then we need to multiply by the inverse of the determinant mod 999.

```
>> Mdet=det(M)

Mdet =
   -34139

>> invmodn(Mdet,999)

ans =
   589
```

The answer is given by

```
>> mod(Minv*589*Mdet,999)

ans =
    772    472    965
    641    516    851
    150    133    149
```

Therefore, the inverse matrix mod 999 is

$$\begin{matrix} 772 & 472 & 965 \\ 641 & 516 & 851 \\ 150 & 133 & 149 \end{matrix} .$$

In many cases, it is possible to determine by inspection the common denominator that must be removed. When this is not the case, note that the determinant of the original matrix will always work as a common denominator.

Example 16

Find a square root of 26951623672 mod the prime $p = 98573007539$.

SOLUTION

Since $p \equiv 3 \pmod 4$, we can use the proposition of Section 3.9:

```
>> powermod(sym('26951623672'),
(sym('98573007539')+1)/4,sym('98573007539'))

ans =
   98338017685
```

The other square root is minus this one:

```
>> mod(-ans,32579)
```

```
ans =
   234989854
```

Example 17

Let $n = 34222273 = 9803 \cdot 3491$. Find all four solutions of $x^2 \equiv 19101358 \pmod{34222273}$.

SOLUTION

First, find a square root mod each of the two prime factors, both of which are congruent to $3 \pmod 4$:

```
>> powermod(19101358,(9803+1)/4,9803)

ans =
   3998
>> powermod(19101358,(3491+1)/4,3491)

ans =
   1318
```

Therefore, the square roots are congruent to $\pm 3998 \pmod{9803}$ and are congruent to $\pm 1318 \pmod{3491}$. There are four ways to combine these using the Chinese remainder theorem:

```
>> crt([3998 1318],[9803 3491])

ans =
   43210

>> crt([-3998 1318],[9803 3491])

ans =
   8397173

>> crt([3998 -1318],[9803 3491])

ans =
   25825100
```

```
>> crt([-3998 -1318],[9803 3491])

ans =
   34179063
```

These are the four desired square roots.

C.4 Examples for Chapter 5

Example 18

Compute the first 50 terms of the recurrence

$$x_{n+5} \equiv x_n + x_{n+2} \pmod 2.$$

The initial values are $0, 1, 0, 0, 0$.

SOLUTION

The vector of coefficients is $[1, 0, 1, 0, 0]$ and the initial values are given by the vector $[0, 1, 0, 0, 0]$. Type

```
>> lfsr([1 0 1 0 0],[0 1 0 0 0],50)

ans =
   Columns 1 through 12
    0   1   0   0   0   0   1   0   0   1   0   1
   Columns 13 through 24
    1   0   0   1   1   1   1   1   0   0   0   1
   Columns 25 through 36
    1   0   1   1   1   0   1   0   1   0   0   0
   Columns 37 through 48
    0   1   0   0   1   0   1   1   0   0   1   1
   Columns 49 through 50
    1   1
```

Example 19

Suppose the first 20 terms of an LFSR sequence are 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1,

0, 1. Find a recursion that generates this sequence.

SOLUTION

First, we find a candidate for the length of the recurrence. The command *lfsrlength(v, n)* calculates the determinants mod 2 of the first n matrices that appear in the procedure described in Section 5.2 for the sequence v. Recall that the last nonzero determinant gives the length of the recurrence.

```
>> lfsrlength([1 0 1 0 1 1 1 0 0 0 0 1 1 1 0 1 0
1 0 1],10)
    Order Determinant
       1        1
       2        1
       3        0
       4        1
       5        0
       6        1
       7        0
       8        0
       9        0
      10        0
```

The last nonzero determinant is the sixth one, so we guess that the recurrence has length 6. To find the coefficients:

```
>> lfsrsolve([1 0 1 0 1 1 1 0 0 0 0 1 1 1 0 1 0 1
0 1],6)

ans =
    1  0  1  1  1  0
```

This gives the recurrence as

$$x_{n+6} \equiv x_n + x_{n+2} + x_{n+3} + x_{n+4} \pmod 2.$$

Example 20

The ciphertext 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0 was produced by adding the output of a LFSR onto the plaintext mod 2 (i.e., XOR the plaintext with the LFSR output). Suppose you know that the plaintext starts 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0. Find the rest of the plaintext.

SOLUTION

XOR the ciphertext with the known part of the plaintext to obtain the beginning of the LFSR output:

```
>> x=mod([1 1 1 1 1 1 0 0 0 0 0 0 1 1 1 0 0]+[0 1
1 0 1 0 1 0 1 0 0 1 1 0 0 0 1],2)

x =
   Columns 1 through 12
    1   0   0   1   0   1   1   0   1   0   0   1
   Columns 13 through 17
    0   1   1   0   1
```

This is the beginning of the LFSR output. Let’s find the length of the recurrence:

```
>> lfsrlength(x,8)
    Order  Determinant
      1         1
      2         0
      3         1
      4         0
      5         1
      6         0
      7         0
      8         0
```

We guess the length is 5. To find the coefficients of the recurrence:

```
>> lfsrsolve(x,5)

ans =
    1   1   0   0   1
```

Now we can generate the full output of the LFSR using the coefficients we just found plus the first five terms of the LFSR output:

```
>> lfsr([1 1 0 0 1],[1 0 0 1 0],40)

ans =
   Columns 1 through 12
    1   0   0   1   0   1   1   0   1   0   0   1
   Columns 13 through 24
    0   1   1   0   1   0   0   1   0   1   1   0
   Columns 25 through 36
    1   0   0   1   0   1   1   0   1   0   0   1
   Columns 37 through 40
    0   1   1   0
```

When we XOR the LFSR output with the ciphertext, we get back the plaintext:

```
>> mod(ans+[0 1 1 0 1 0 1 0 1 0 0 1 1 0 0 0 1 0 1
0 1 0 1 0 1 0 1 0 1 0 0 1 0 0 0 1 0 1 1 0],2)

ans =
   Columns 1 through 12
    1   1   1   1   1   1   0   0   0   0   0   0
   Columns 13 through 24
    1   1   1   0   0   0   1   1   1   1   0   0
   Columns 25 through 36
    0   0   1   1   1   1   1   1   1   0   0   0
   Columns 37 through 40
    0   0   0   0
```

This is the plaintext.

C.5 Examples for Chapter 6

Example 21

The ciphertext

$$22, 09, 00, 12, 03, 01, 10, 03, 04, 08, 01, 17$$

was encrypted using a Hill cipher with matrix

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{pmatrix}.$$

Decrypt it.

SOLUTION

A matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ is entered as $[a, b; c, d]$. Type $M * N$ to multiply matrices M and N. Type $v * M$ to multiply a vector v on the right by a matrix M.

First, we put the above matrix in the variable M.

```
>> M=[1 2 3; 4 5 6; 7 8 10]

M =
```

$$\begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{matrix}$$

Next, we need to invert the matrix mod 26:

```
>> Minv=inv(M)

   Minv =
```

$$\begin{matrix} -2/3 & -4/3 & 1 \\ -2/3 & 11/3 & -2 \\ 1 & -2 & 1 \end{matrix}$$

Since we are working mod 26, we can't stop with numbers like $2/3$. We need to get rid of the denominators and reduce mod 26. To do so, we multiply by 3 to extract the numerators of the fractions, then multiply by the inverse of 3 mod 26 to put the "denominators" back in (see Section 3.3):

```
>> M1=Minv*3

M1 =
```

$$\begin{matrix} -2 & -4 & 3 \\ -2 & 11 & -6 \\ 3 & -6 & 3 \end{matrix}$$

```
>> M2=mod(round(M1*9,26))

M2 =
```

$$\begin{matrix} 8 & 16 & 1 \\ 8 & 21 & 24 \\ 1 & 24 & 1 \end{matrix}$$

Note that we used the function *round* in calculating $M2$. This was done since MATLAB performs its calculations in floating point and calculating the inverse matrix $Minv$ produces numbers that are slightly different from whole numbers. For example, consider the following:

```
>> a=1.99999999;display([a, mod(a,2),
mod(round(a),2)])
```

```
2.0000 2.0000 0
```

The matrix $M2$ is the inverse of the matrix M mod 26. We can check this as follows:

```
>> mod(M2*M,26)
ans =
    1   0   0
    0   1   0
    0   0   1
```

To decrypt, we break the ciphertext into blocks of three numbers and multiply each block on the right by the inverse matrix we just calculated:

```
>> mod([22,9,0]*M2,26)

ans =
    14   21   4

>> mod([12,3,1]*M2,26)

ans =
    17   19   7

>> mod([10,3,4]*M2,26)

ans =
    4   7   8
```

```
>> mod([8,1,17]*M2,26)

ans =
    11   11   23
```

Therefore, the plaintext is 14, 21, 4, 17, 19, 7, 4, 7, 8, 11, 11, 23. This can be changed back to letters:

```
>> int2text([14 21 4 17 19 7 4 7 8 11 11 23])

ans =

overthehillx
```

Note that the final x was appended to the plaintext in order to complete a block of three letters.

C.6 Examples for Chapter 9

Example 22

Two functions, *nextprime* and *randprime*, can be used to generate prime numbers. The function *nextprime* takes a number n as input and attempts to find the next prime after n. The function *randprime* takes a number n as input and attempts to find a random prime between 1 and n. It uses the Miller-Rabin test described in Chapter 9.

```
>> nextprime(346735)

ans =
   346739

>> randprime(888888)

ans =
   737309
```

For larger inputs, use symbolic mode:

```
>> nextprime(10^sym(60))

ans =

1000000000000000000000000000000000000000000000000
000000000007

>> randprime(10^sym(50))

ans =
```

```
582325165358256624514865507080685347318649291992
19
```

It is interesting to note the difference that the ’ ’ makes when entering a large integer:

```
>>
nextprime(sym(’12345678901234567890123456789O’))

ans =
   123456789012345678901234567907

>> nextprime(sym(123456789012345678901234567890))

ans =
   123456789012345677877719597071
```

In the second case, the input was a number, so only the first 16 digits of the input were used correctly when changing to symbolic mode, while the first case regarded the entire input as a string and therefore used all of the digits.

Example 23

Suppose you want to change the text `hellohowareyou` to numbers:

```
>> text2int1(’hellohowareyou’)

ans =
   805121215081523011805251521
```

Note that we are now using $a = 1$, $b = 2$, ..., $z = 26$, since otherwise a’s at the beginnings of messages would disappear. (A more efficient procedure would be to work in base 27, so the numerical form of the message would be

$8 + 5 \cdot 27 + 12 \cdot 27^2 + \cdots + 21 \cdot 27^{13} = 87495221502384554951$
. Note that this uses fewer digits.)

Now suppose you want to change it back to letters:

```
>> int2text1(805121215081523011805251521)

ans =
   'hellohowareyou'
```

Example 24

Encrypt the message `hi` using RSA with $n = 823091$ and $e = 17$.

SOLUTION

First, change the message to numbers:

```
>> text2int1('hi')

ans =
   809
```

Now, raise it to the eth power mod n:

```
>> powermod(ans,17,823091)

ans =
   596912
```

Example 25

Decrypt the ciphertext in the previous problem.

SOLUTION

First, we need to find the decryption exponent d. To do this, we need to find $\phi(823091)$. One way is

```
>> eulerphi(823091)

ans =
   821184
```

Another way is to factor n as $p \cdot q$ and then compute $(p-1)(q-1)$:

```
>> factor(823091)

ans =
    659   1249

>> 658*1248

ans =
   821184
```

Since $de \equiv 1 \pmod{\phi(n)}$, we compute the following (note that we are finding the inverse of e mod $\phi(n)$, not mod n):

```
>> invmodn(17,821184)

ans =
   48305
```

Therefore, $d = 48305$. To decrypt, raise the ciphertext to the dth power mod n:

```
>> powermod(596912,48305,823091)

ans =
   809
```

Finally, change back to letters:

```
>> int2text1(ans)

ans =
   hi
```

Example 26

Encrypt `hellohowareyou` using RSA with $n = 823091$ and $e = 17$.

SOLUTION

First, change the plaintext to numbers:

```
>> text2int1('hellohowareyou')

ans =
   805121215081523011805251521
```

Suppose we simply raised this to the eth power mod n:

```
>> powermod(ans,17,823091)

ans =
   447613
```

If we decrypt (we know d from Example 25), we obtain

```
>> powermod(ans,48305,823091)

ans =
   628883
```

This is not the original plaintext. The reason is that the plaintext is larger than n, so we have obtained the plaintext mod n:

```
>> mod(text2int1('hellohowareyou'),823091)

ans =
   628883
```

We need to break the plaintext into blocks, each less than n. In our case, we use three letters at a time:

$$80512 \quad 121508 \quad 152301 \quad 180525 \quad 1521$$

```
>> powermod(80512,17,823091)

ans =
   757396

>> powermod(121508,17,823091)

ans =
   164513

>> powermod(152301,17,823091)

ans =
   121217

>> powermod(180525,17,823091)

ans =
   594220

>> powermod(1521,17,823091)

ans =
   442163
```

The ciphertext is therefore 757396164513121217594220442163. Note that there is no reason to change this back to letters. In fact, it doesn't correspond to any text with letters.

Decrypt each block individually:

```
>> powermod(757396,48305,823091)

ans =
   80512

>> powermod(164513,48305,823091)

ans =
   121508
```

Etc.

We'll now do some examples with large numbers, namely the numbers in the RSA Challenge discussed in Section 9.5. These are stored under the names *rsan, rsae, rsap, rsaq*:

```
>> rsan

ans =

1143816257578888676692357799761466120102182967212
42362562561842935
7069352457338978305971235639587050589890751475992
90026879543541

>> rsae

ans =
   9007
```

Example 27

Encrypt each of the messages *b, ba, bar, bard* using *rsan* and *rsae*.

```
>> powermod(text2int1('b'), rsae, rsan)

ans =

7094675846761266859837016499155078618287633106068
52354105647041144
```

```
867822617164972001221553323484620140532879875808
9263765142534

>> powermod(text2int1('ba'), rsae, rsan)

ans =

350451306089751003250117094498719542737882047539
485930603136976982
276217598060279622705380315655647733520336717822
61305796158951

>> powermod(txt2int1('bar'), rsae, rsan)

ans =

448145128638551010760045308594921093424295316066
074090703605434080
008436459868804059531028183128225863625802987844
41151922606424

>> powermod(text2int1('bard'), rsae, rsan)

ans =

242380777851116664232028625120903173934852129590
562707831349916142
560543232971798049289580734457526630264498739868
77989329909498
```

Observe that the ciphertexts are all the same length. There seems to be no easy way to determine the length of the corresponding plaintext.

Example 28

Using the factorization $rsan = rsap \cdot rsaq$, find the decryption exponent for the RSA Challenge, and decrypt the ciphertext (see Section 9.5).

SOLUTION

First, we find the decryption exponent:

```
>> rsad=invmodn(rsae,-1,(rsap-1)*(rsaq-1));
```

Note that we use the final semicolon to avoid printing out the value. If you want to see the value of *rsad*, see Section 9.5, or don't use the semicolon. To decrypt the ciphertext, which is stored as *rsaci*, and change to letters:

```
>> int2text1(powermod(rsaci, rsad, rsan))

ans =
   the magic words are squeamish ossifrage
```

Example 29

Encrypt the message *rsaencryptsmessageswell* using *rsan* and *rsae*.

```
>> ci =
powermod(text2int1('rsaencryptsmessageswell'),
rsae, rsan)
   ci =

9463942034900225931630582353924949641464096993400
17097214043524182
7195065425436558490601396632881775353928311265319
7553130781884
```

We called the ciphertext `ci` because we need it in Example 30.

Example 30

Decrypt the preceding ciphertext.

SOLUTION

Fortunately, we know the decryption exponent *rsad*. Therefore, we compute

```
>> powermod(ans, rsad, rsan)

ans =
   18190105140318251620191305191901070519230512 12

>> int2text1(ans)

ans =
   rsaencryptsmessageswell
```

Suppose we lose the final 4 of the ciphertext in transmission. Let's try to decrypt what's left (subtracting 4 and dividing by 10 is a mathematical way to remove the 4): ‘ `>> powermod((ci - 4)/10, rsad, rsan)`

```


ans =

47952999173195988664902352629525486409113633894375629846854907970588412300373487969657794254117158956921267912628461494475682806
```

If we try to change this to letters, we get a weird-looking answer. A small error in the plaintext completely changes the decrypted message and usually produces garbage.

Example 31

Suppose we are told that $n = 11313771275590312567$ is the product of two primes and that $\phi(n) = 11313771187608744400$. Factor n.

SOLUTION

We know (see Section 9.1) that p and q are the roots of $X^2 - (n - \phi(n) + 1)X + n$. Therefore, we compute (vpa is for variable precision arithmetic)

```
>> digits(50); syms y; vpasolve(y^2-
(sym('11313771275590312567') -
sym('11313771187608744400')+1)*y+sym('11313771275
590312567'),y)

ans =
   128781017.0 87852787151.0
```

Therefore, $n = 128781017 \cdot 87852787151$. We also could have used the quadratic formula to find the roots.

Example 32

Suppose we know *rsae* and *rsad*. Use these to factor *rsan*.

SOLUTION

We use the $a^r \equiv 1$ factorization method from Section 9.4. First write $rsae \cdot rsad - 1 = 2^s m$ with m odd. One way to do this is first to compute

```
>> rsae*rsad - 1

ans =

961034419617782266156919023359583834109854129051878330250644604
0
411559855750873526591561748985573429951315946804310869212458300
97664
```

```
>> ans/2

ans =

480517209808891133078459511679791917054927064525 9
391651253223020
205779927875436763295780874492786714975657973402 1
5543460622915048832

>> ans/2

ans =

240258604904445566539229755839895958527463532262 9
695825626611510
102889963937718381647890437246393357487828986701 0
7771730311457524416
```

We continue this way for six more steps until we get

```
ans =

375404070163196197717546493499837435199161769160 8
899727541580484
535765568652684971324828808197489621074732791720 4
33933286116523819
```

This number is m. Now choose a random integer a. Hoping to be lucky, we choose 13. As in the $a^r \equiv 1$ factorization method, we compute

```
>>b0=powermod(13, ans, rsan)

b0 =

275743685070065305922434948688471611984230957073 0
780569056983964
703018310983986237080052933809298479549019264358 7
960859870551239
```

Since this is not $\pm 1 \pmod{rsan}$, we successively square it until we get ± 1:

```
>> b1=powermod(b0,2,rsan)

b1 =

48318960321928515580138476418723034554104099069940
84622549470277
66549964125829556360352661561086864311942985740758
54037512277292

>> b2=powermod(b1,2,rsan)

b2 =

78172814154877356579141928058754000021948787056483
82091793062511
52151818397420560132755219134875609447320735164877
22273875579363

>> b3=powermod(b2, 2, rsan)

b3 =

42836191202508728742199299040582900202976222916017
76716755187021
65094445182394621863794705694420551013929922930822
59601738228702

>> b4=powermod(b3, 2, rsan)

b4 =
   1
```

Since the last number before the 1 was not $\pm 1 \pmod{rsan}$, we have an example of $x \not\equiv \pm 1 \pmod{rsan}$ with $x^2 \equiv 1$. Therefore, $\gcd(x-1,\ rsan)$ is a nontrivial factor of $rsan$:

```
>> gcd(b3 - 1, rsan)

ans =

32769132993266709549961988190834461413177642967992942539798288533
```

This is *rsaq*. The other factor is obtained by computing *rsan/rsaq*:

```
>> rsan/ans

ans =

3490529510847650949147849619903898133417764638493
387843990820577
```

This is *rsap*.

Example 33

Suppose you know that

$$150883475569451^2 \equiv 16887570532858^2 \pmod{205611444308117}.$$

Factor 205611444308117.

SOLUTION

We use the Basic Principle of Section 9.4.

```
>> g= gcd(150883475569451-
16887570532858,205611444308117)

g =
   23495881
```

This gives one factor. The other is

```
>> 205611444308117/g

ans =
   8750957
```

We can check that these factors are actually primes, so we can't factor any further:

```
>> primetest(ans)

ans =
   1

>> primetest(g)

ans =
   1
```

Example 34

Factor
$n = 37687557542639485559998999 2897873239$ by the $p - 1$ method.

SOLUTION

Let's choose our bound as $B = 100$, and let's take $a = 2$, so we compute $2^{100!} \pmod{n}$:

```
>>
powermod(2,factorial(100),sym('376875575426394855
59998999 2897873239')

ans =
   369676678301956331939422106251199512
```

Then we compute the gcd of $2^{100!} - 1$ and n:

```
>> gcd(ans - 1,
sym'(37687557542639485559998999 2897873239')

ans =
   430553161739796481
```

This is a factor p. The other factor q is

```
>>
sym('37687557542639485559998999 2897873239')/ans
```

```
ans =
   875328783798732119
```

Let's see why this worked. The factorizations of $p-1$ and $q-1$ are

```
>> factor(sym('430553161739796481') - 1)

ans =
   [ 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 5, 7, 7, 7, 7, 11,
11, 11, 47]

>> factor(sym('875328783798732119') - 1)

ans =
   [ 2, 61, 20357, 39301, 8967967]
```

We see that $100!$ is a multiple of $p-1$, so $2^{100!} \equiv 1 \pmod{p}$. However, $100!$ is not a multiple of $q-1$, so it is likely that $2^{100!} \not\equiv 1 \pmod{q}$. Therefore, both $2^{100!}-1$ and pq have p as a factor, but only pq has q as a factor. It follows that the gcd is p.

C.7 Examples for Chapter 10

Example 35

Let's solve the discrete log problem $2^x \equiv 71 \pmod{131}$ by the Baby Step-Giant Step method of Subsection 10.2.2. We take $N = 12$ since $N^2 > p - 1 = 130$ and we form two lists. The first is $2^j \pmod{1}31$ for $0 \leq j \leq 11$:

```
>> for k=0:11;z=[k,
powermod(2,k,131)];disp(z);end;

0 1
1 2
2 4
3 8
4 16
5 32
6 64
7 128
8 125
9 119
10 107
11 83
```

The second is $71 \cdot 2^{-j} \pmod{1}31$ for $0 \leq j \leq 11$:

```
>> for k=0:11;z=[k,
mod(71*invmodn(powermod(2,12*k,131),131),131)];
disp(z);end;

0 71
1 17
2 124
3 26
4 128
```

```
5 86
6 111
7 93
8 85
9 96
10 130
11 116
```

The number 128 is on both lists, so we see that $2^7 \equiv 71 \cdot 2^{-12*4} \pmod{131}$. Therefore,

$$71 \equiv 2^{7+4*12} \equiv 2^{55} \pmod{131}.$$

C.8 Examples for Chapter 12

Example 36

Suppose there are 23 people in a room. What is the probability that at least two have the same birthday?

SOLUTION

The probability that no two have the same birthday is $\prod_{i=1}^{22}(1 - i/365)$ (note that the product stops at $i = 22$, not $i = 23$). Subtracting from 1 gives the probability that at least two have the same birthday:

```
>> 1-prod( 1 - (1:22)/365)

ans =
   0.5073
```

Example 37

Suppose a lazy phone company employee assigns telephone numbers by choosing random seven-digit numbers. In a town with 10,000 phones, what is the probability that two people receive the same number?

```
>> 1-prod( 1 - (1:9999)/10^7)
```

```
ans =
   0.9933
```

Note that the number of phones is about three times the square root of the number of possibilities. This means that we expect the probability to be high, which it is. From Section 12.1, we have the estimate that if there are around $\sqrt{2(\ln 2)10^7} \approx 3723$ phones, there should be a 50% chance of a match. Let's see how accurate this is:

```
>> 1-prod( 1 - (1:3722)/10^7)

ans =
   0.4999
```

C.9 Examples for Chapter 17

Example 38

Suppose we have a (5, 8) Shamir secret sharing scheme. Everything is mod the prime $p = 987541$. Five of the shares are

$(9853, 853), (4421, 4387), (6543, 1234), (93293, 78428), (12398, 7563).$

Find the secret.

SOLUTION

The function *interppoly(x,f,m)* calculates the interpolating polynomial that passes through the points (x_j, f_j). The arithmetic is done mod m.

In order to use this function, we need to make a vector that contains the x values, and another vector that contains the share values. This can be done using the following two commands:

```
>> x=[9853 4421 6543 93293 12398];

>> s=[853 4387 1234 78428 7563];
```

Now we calculate the coefficients for the interpolating polynomial.

```
>> y=interppoly(x,s,987541)

y =
    678987   14728   1651   574413   456741
```

The first value corresponds to the constant term in the interpolating polynomial and is the secret value. Therefore, 678987 is the secret.

C.10 Examples for Chapter 18

Example 39

Here is a game you can play. It is essentially the simplified version of poker over the telephone from Section 18.2. There are five cards: `ten, jack, queen, king, ace`. We have chosen to abbreviate them by the following: ten, ace, que, jac, kin. They are shuffled and disguised by raising their numbers to a random exponent mod the prime 300649. You are supposed to guess which one is the ace.

First, the cards are entered in and converted to numerical values by the following steps:

```
>> cards=['ten';'ace';'que';'jac';'kin'];

>> cvals=text2int1(cards)

cvals =

   200514
   10305
   172105
   100103
   110914
```

Next, we pick a random exponent k that will be used in the hiding operation. We use the semicolon after *khide* so that we cannot cheat and see what value of k is being used.

```
>> p=300649;
```

```
>> k=khide(p);
```

Now, shuffle the disguised cards (their numbers are raised to the kth power mod p and then randomly permuted):

```
>> shufvals=shuffle(cvals,k,p)

shufvals =
    226536
    226058
    241033
    281258
    116809
```

These are the five cards. None looks like the ace; that's because their numbers have been raised to powers mod the prime. Make a guess anyway. Let's see if you're correct.

```
>> reveal(shufvals,k,p)

ans =


jac
que
ten
kin
ace
```

Let's play again:

```
>> k=khide(p);

» shufvals=shuffle(cvals,k,p)

shufvals =
    117135
```

```
    144487
    108150
    266322
    264045
```

Make your guess (note that the numbers are different because a different random exponent was used). Were you lucky?

```
>> reveal(shufvals,k,p)

ans =

kin
jac
ten
que
ace
```

Perhaps you need some help. Let’s play one more time:

```
>> k=khide(p);

» shufvals=shuffle(cvals,k,p)

shufvals =
    108150
    144487
    266322
    264045
    117135
```

We now ask for advice:

```
>> advise(shufvals,p);
```

```
Ace Index: 4
```

We are advised that the fourth card is the ace. Let’s see:

```
>> reveal(shufvals,k,p)

ans =


ten
jac
que
ace
kin
```

How does this work? Read the part on “How to Cheat” in Section 18.2. Note that if we raise the numbers for the cards to the $(p-1)/2$ power mod p, we get

```
>> powermod(cvals,(p-1)/2,p)

ans =
       1
  300648
       1
       1
       1
```

Therefore, only the ace is a quadratic nonresidue mod p.

C.11 Examples for Chapter 21

Example 40

We want to graph the elliptic curve $y^2 = x(x-1)(x+1)$.

First, we create a string v that contains the equation we wish to graph.

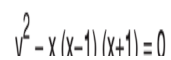
```
>> v='y^2 - x*(x-1)*(x+1)';
```

Next we use the *ezplot* command to plot the elliptic curve.

```
>> ezplot(v,[-1,3,-5,5])
```

The plot appears in Figure C.1. The use of $[-1, 3, -5, 5]$ in the preceding command is to define the limits of the x-axis and y-axis in the plot.

Figure C.1 Graph of the Elliptic Curve $y^2 = x(x-1)(x+1)$.



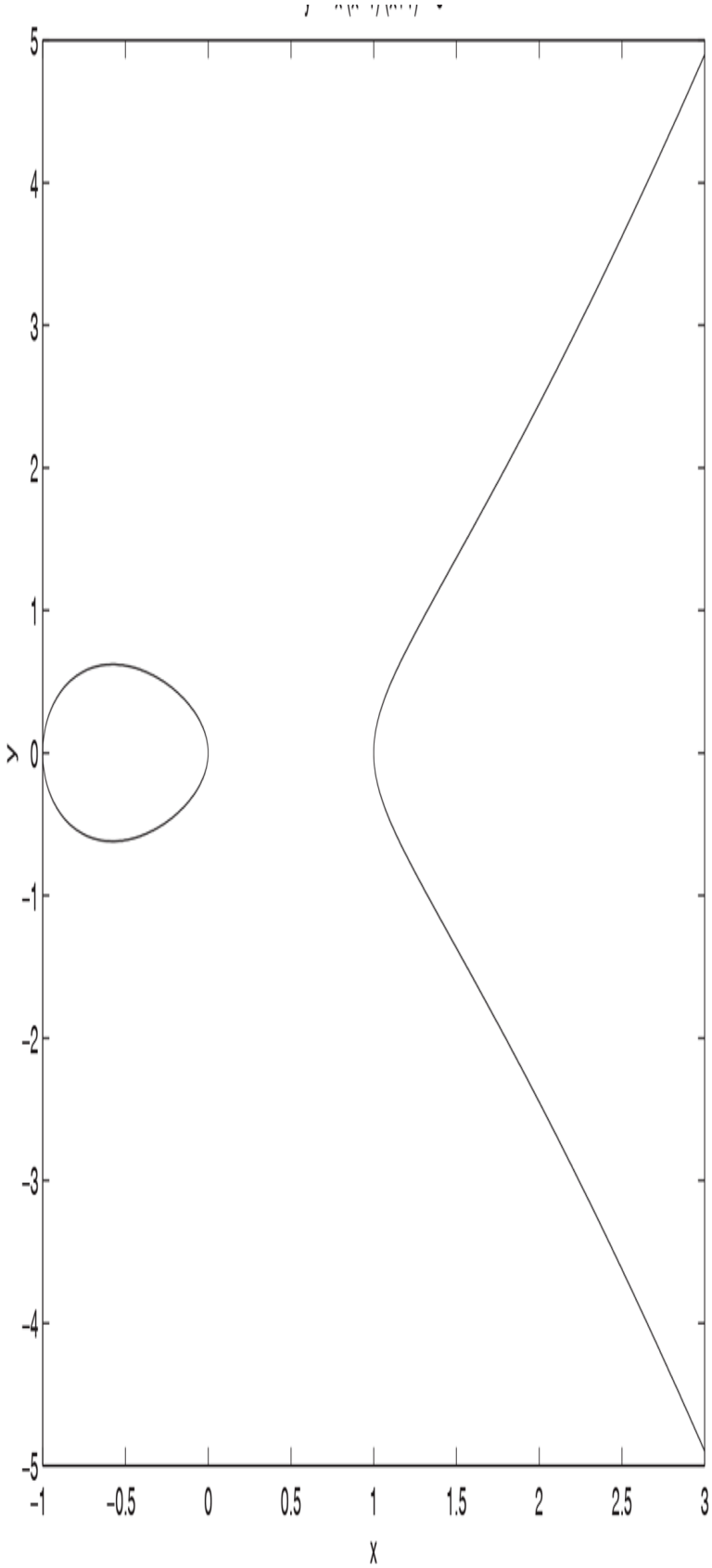


Figure C.1 Full Alternative Text

Example 41

Add the points (1,3) and (3,5) on the elliptic curve $y^2 \equiv x^3 + 24x + 13 \pmod{29}$.

```
>> addell([1,3],[3,5],24,13,29)

ans =
    26    1
```

You can check that the point (26,1) is on the curve: $26^3 + 24 \cdot 26 + 13 \equiv 1^2 \pmod{29}$. (*Note:* `addell([x,y],[u,v],b,c,n)` is only programmed to work for odd n.)

Example 42

Add (1,3) to the point at infinity on the curve of the previous example.

```
>> addell([1,3],[inf,inf],24,13,29)

ans =
    1    3
```

As expected, adding the point at infinity to a point P returns the point P.

Example 43

Let $P = (1, 3)$ be a point on the elliptic curve $y^2 \equiv x^3 + 24x + 13 \pmod{29}$. Find $7P$.

```
>> multell([1,3],7,24,13,29)

ans =
    15    6
```

Example 44

Find $k(1, 3)$ for $k = 1, 2, 3, \ldots, 40$ on the curve of the previous example.

```
>> multsell([1,3],40,24,13,29)

ans =
   1:    1     3
   2:    11    10
   3:    23    28
   4:    0     10
   5:    19    9
   6:    18    19
   7:    15    6
   8:    20    24
   9:    4     12
  10:    4     17
  11:    20    5
  12:    15    23
  13:    18    10
  14:    19    22
  15:    0     19
  16:    23    1
  17:    11    19
  18:    1     26
  19:   inf   Inf
  20:    1     3
  21:    10    10
  22:    23    28
  23:    0     10
  24:    19    7
  25:    18    19
  26:    15    6
  27:    20    24
  28:    4     12
  29:    4     17
  30:    20    5
  31:    15    23
  32:    18    10
  33:    19    22
  34:    0     19
  35:    23    1
  36:    11    19
  37:    1     26
```

```
   38:   inf   inf
   39:    1     3
   40:    10    10
```

Notice how the points repeat after every 19 multiples.

Example 45

The previous four examples worked mod the prime 29. If we work mod a composite number, the situation at infinity becomes more complicated since we could be at infinity mod both factors or we could be at infinity mod one of the factors but not mod the other. Therefore, we stop the calculation if this last situation happens and we exhibit a factor. For example, let's try to compute $12P$, where $P = (1, 3)$ is on the elliptic curve $y^2 \equiv x^3 - 5x + 13 \pmod{209}$:

```
>> multell([1,3],12,-5,13,11*19)

Elliptic Curve addition produced a factor of n,
factor= 19
Multell found a factor of n and exited

ans =
   []
```

Now let's compute the successive multiples to see what happened along the way:

```
>> multsell([1,3],12,-5,13,11*19)

Elliptic Curve addition produced a factor of n,
factor= 19
Multsell ended early since it found a factor

ans =
1:   1     3
```

```
 2:   91    27
 3:   118   133
 4:   148   182
 5:   20    35
```

When we computed $6P$, we ended up at infinity mod 19. Let's see what is happening mod the two prime factors of 209, namely 19 and 11:

```
>> multsell([1,3],20,-5,13,19)

ans =
 1:    1     3
 2:   15     8
 3:    4     0
 4:   15    11
 5:    1    16
 6:   Inf   Inf
 7:    1     3
 8:   15     8
 9:    4     0
10:   15    11
11:   15     8
12:   Inf   Inf
13:    1     3
14:   15     8
15:    4     0
16:   15    11
17:    1    16
18:   Inf   Inf
19:    1     3
20:   15     8
```

```
>> multsell([1,3],20,-5,13,11)

ans =
 1:    1     3
 2:    3     5
 3:    8     1
 4:    5     6
 5:    9     2
 6:    6    10
 7:    2     0
 8:    6     1
 9:    9     9
10:    5     5
11:    8    10
```

```
12:    3     6
13:    1     8
14:   Inf   Inf
15:    1     3
16:    3     5
17:    8     1
18:    5     6
19:    9     2
20:    6    10
```

After six steps, we were at infinity mod 19, but it takes 14 steps to reach infinity mod 11. To find $6P$, we needed to invert a number that was 0 mod 19 and nonzero mod 11. This couldn't be done, but it yielded the factor 19. This is the basis of the elliptic curve factorization method.

Example 46

Factor 193279 using elliptic curves.

SOLUTION

First, we need to choose some random elliptic curves and a point on each curve. For example, let's take $P = (2, 4)$ and the elliptic curve

$$y^2 \equiv x^3 - 10x + b \pmod{193279}.$$

For P to lie on the curve, we take $b = 28$. We'll also take

$$\begin{aligned} y^2 &\equiv x^3 + 11x - 11, \quad P = (1, 1), \\ y^2 &\equiv x^3 + 17x - 14, \quad P = (1, 2). \end{aligned}$$

Now we compute multiples of the point P. We do the analog of the $p - 1$ method, so we choose a bound B, say $B = 12$, and compute $B!P$.

```
>> multell([2,4],factorial(12),-10,28,193279)
```

```
Elliptic Curve addition produced a factor of n,
factor= 347
Multell found a factor of n and exited

ans =
   []

>> multell([1,1],factorial(12),11,-11,193279)

ans =
    13862    35249

» multell([1,2],factorial(12),17,-14,193279)

Elliptic Curve addition produced a factor of n,
factor= 557
Multell found a factor of n and exited

ans =
   []
```

Let’s analyze in more detail what happened in these examples.

On the first curve, $266P$ ends up at infinity mod 557 and $35P$ is infinity mod 347. Since $272 = 2 \cdot 7 \cdot 9$, it has a prime factor larger than $B = 12$, so $B!P$ is not infinity mod 557. But 35 divides $B!$, so $B!P$ is infinity mod 347.

On the second curve, $356P =$ infinity mod 347, and $561P =$ infinity mod 557. Since $356 = 4 \cdot 89$ and $561 = 3 \cdot 11 \cdot 17$, we don’t expect to find the factorization with this curve.

The third curve is a surprise. We have $331P =$ infinity mod 347 and $272P =$ infinity mod 557. Since 331 is prime and $272 = 16 \cdot 17$, we don’t expect to find the factorization with this curve. However, by chance, an intermediate step in the calculation of $B!P$ yielded the factorization. Here’s what happened. At an intermediate step in the calculation, the program

required adding the points $(184993, 13462)$ and $(20678, 150484)$. These two points are congruent mod 557 but not mod 347. Therefore, the slope of the line through these two points is defined mod 347 but is $0/0$ mod 557. When we tried to find the multiplicative inverse of the denominator mod 193279, the gcd algorithm yielded the factor 557. This phenomenon is fairly rare.

Example 47

Here is how to produce the example of an elliptic curve ElGamal cryptosystem from Section 21.5. For more details, see the text. The elliptic curve is $y^2 \equiv x^3 + 3x + 45 \pmod{8831}$ and the point is $G = (4, 11)$. Alice's message is the point $P_m = (5, 1743)$.

Bob has chosen his secret random number $a_B = 3$ and has computed $a_B G$:

```
>> multell([4,11],3,3,45,8831)

ans =
    413    1808
```

Bob publishes this point. Alice chooses the random number $k = 8$ and computes kG and $P_m + k(a_B G)$:

```
>> multell([4,11],8,3,45,8831)

ans =
    5415    6321

>>
addell([5,1743],multell([413,1808],8,3,45,8831),3
,45,8831)
```

```
ans =
    6626    3576
```

Alice sends (5415,6321) and (6626, 3576) to Bob, who multiplies the first of these point by a_B:

```
>> multell([5415,6321],3,3,45,8831) ans = 673 146
```

Bob then subtracts the result from the last point Alice sends him. Note that he subtracts by adding the point with the second coordinate negated:

```
>> addell([6626,3576],[673,-146],3,45,8831)

ans =
   5 1743
```

Bob has therefore received Alice's message.

Example 48

Let's reproduce the numbers in the example of a Diffie-Hellman key exchange from Section 21.5: The elliptic curve is $y^2 \equiv x^3 + x + 7206 \pmod{7211}$ and the point is $G = (3, 5)$. Alice chooses her secret $N_A = 12$ and Bob chooses his secret $N_B = 23$. Alice calculates

```
>> multell([3,5],12,1,7206,7211)

ans =
    1794    6375
```

She sends (1794,6375) to Bob. Meanwhile, Bob calculates

```
>> multell([3,5],23,1,7206,7211)
```

```
ans =
    3861    1242
```

and sends (3861,1242) to Alice. Alice multiplies what she receives by N_A and Bob multiplies what he receives by N_B:

```
>> multell([3861,1242],12,1,7206,7211)

ans =
    1472    2098

>> multell([1794,6375],23,1,7206,7211)

ans =
    1472    2098
```

Therefore, Alice and Bob have produced the same key.